

Hunting In Memory (Lab 1)

LAB 7.1

Scenario

The organization you work for is asking you to perform memory threat hunting on a randomly selected machine. The IT Security manager tasked you specifically with looking for anomalous connections and memory injections.

A memory image has already been created and is waiting for you!

Goals

The learning objective of this lab is to get more familiar and comfortable with Volatility.

What you will learn

You will learn how to utilize a memory dump to confirm a compromise.

Recommended tools

- Volatility
- Strings
- Base64

Tasks

Task 1. Are there any hidden processes?

The machine's memory image file can be found in the */root/* directory.

Begin by identifying the image profile that is required by Volatility before it can parse the memory file.

It is very common for rootkits to hide the presence of a process from tools such as Task Manager on a live Windows machine. However, with the use of Volatility's *psscan/psxview* plugins, we can identify anomalies that may exist.

Task 2. Hunt for suspicious network connections?

Use Volatility's *netscan* plugin and review the output for anything that looks suspicious.

Task 3. Hunt for process injection

Use Volatility's *malfind* plugin and review the output for anything that looks suspicious. Be aware of false-positives, as certain processes perform injection as part of their regular operation.

Task 4. What can we conclude?

Based on the information discovered, what can you conclude and how can you extract additional information that will conclude your hunt and be useful to other teams such as Incident Responders.

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Are there any hidden processes?

To begin with, we can execute Volatility's *imageinfo* plugin, to get an idea of the profile of the machine. Let's do so by navigating to */root/* and executing the command below.

```
vol.py -f /root/Memory.vmem imageinfo
```

In the output, Volatility displays the following:

```
root@attackdefense:~# vol.py -f /root/Memory.vmem
Volatility Foundation Volatility Framework 2.6.
INFO      : volatility.debug      : Determining profile
          : Suggested Profile(s) : Win10x64_17134
10x64_15063 (Instantiated with Win10x64_15063)
          : AS Layer1 : SkipDuplicates
          : AS Layer2 : FileAddressSpace
          : PAE type  : No PAE
          : DTB       : 0x1aa000L
          : KDBG      : 0xf80171b42a60
          : Number of Processors : 2
          : Image Type (Service Pack) : 0
          : KPCR for CPU 0 : 0xffffffff80171b9
          : KPCR for CPU 1 : 0xfffffd000893e
          : KUSER_SHARED_DATA : 0xffffffff7800000
          : Image date and time : 2019-11-18 17:
          : Image local date and time : 2019-11-18 17:
root@attackdefense:~#
```

Unfortunately, Volatility did not recognize the profile immediately, after some trial and error approach, it appears that the correct one is "Win10x64_10586", which is one of the possible profiles that Volatility suggested.

Armed with the profile, we proceed further to look at the output of *psxview*, directly jumping into looking for anomalies related to process hiding. The output of it would look similar as on the image below, where we have cropped the image for brevity:

```
vol.py -f /root/Memory.vmem --profile=Win10x64_10586 psxview
```

```

0x000000001d3e3840 SearchIndexer.      868 True  True  True  True  True  True  False
0x0000000001764840 svchost.exe      1112 True  True  True  True  True  True  False
0x0000000007e270380 System           4 True  True  True  True  False False  False
0x0000000006e4f2340 csrss.exe        448 True  True  True  True  False True  False
0x0000000006e538440 smss.exe          428 True  True  False True  False True  False  2019-11-18 16:40:51 UTC+0000
0x0000000007635e840 smss.exe          272 True  True  True  True  False False  False
0x00000000072986080 csrss.exe          360 True  True  True  True  False True  False
0x000000000721db840 userinit.exe      2148 True  True  False True  False True  False  2019-11-18 16:44:38 UTC+0000
0x0000000003ce4aa20                0 False False False  False False False  True
0x00000000032331120                0 False False False  False False False  True
0x000000000644bdb30 ?/@???/?@???      3 False False False  False False False  True
0x0000000003181f6c0                0 False False False  False False False  True
0x00000000079ac0a20 ??????????      0 False False False  False False False  True
root@attackdefense:~# █

```

Although it appears that there are processes which seem not detected by 6 out of 7 of the techniques that *psxview* uses, they are just false positives. It is likely that the EPROCESS memory structure has persisted from a previous reboot, however they are lacking many of the attributes, that's why Volatility fails to parse them properly.

No running processes are hidden.

Task 2. Hunt for suspicious network connections

To look at network connections, we'll use Volatility's netscan plugin. After running it, we'll notice that multiple PowerShell instances have been making (they are now closed but still preserved in memory!) connections to port 80. Remember, we are generally interested in any process that makes connections to port 80 and 443, which is not a browser. Here is the output from the plugin, specifically filtering out the connections from PowerShell:

```
vol.py -f /root/Memory.vmem --profile=Win10x64 10586 netscan | grep -E 'Pid|powershell.exe'
```

```
root@attackdefense:~# vol.py -f /root/Memory.vmem --profile=Win10x64_10586 netscan | grep -E 'Pid|powershell.exe'
```

Volatility Foundation Volatility Framework 2.6.1

Offset(P)	Proto	Local Address	Foreign Address	State	Pid	Owner	Created
0xe001eb03f010 14 UTC+0000	UDPv4	0.0.0.0:0	*:*		3004	powershell.exe	2019-11-18 16:55:
0xe001eb03f010 14 UTC+0000	UDPv6	:::0	*:*		3004	powershell.exe	2019-11-18 16:55:
0xe001eafb0800 27 UTC+0000	TCPv4	169.254.55.147:50065	169.254.55.148:80	CLOSED	3720	powershell.exe	2019-11-18 17:07:
0xe001eb904ec0 14 UTC+0000	UDPv4	0.0.0.0:0	*:*		3004	powershell.exe	2019-11-18 16:55:
0xe001eb904ec0 14 UTC+0000	UDPv6	:::0	*:*		3004	powershell.exe	2019-11-18 16:55:
0xe001eb9c5e00 14 UTC+0000	UDPv4	0.0.0.0:0	*:*		3004	powershell.exe	2019-11-18 16:55:
0xe001eba429e0 09 UTC+0000	TCPv4	169.254.55.147:50010	169.254.55.148:80	CLOSED	3004	powershell.exe	2019-11-18 17:05:
0xe001eba50580 32 UTC+0000	TCPv4	169.254.55.147:50067	169.254.55.148:80	CLOSED	3720	powershell.exe	2019-11-18 17:07:
0xe001ec2ef010 45 UTC+0000	UDPv4	0.0.0.0:0	*:*		3720	powershell.exe	2019-11-18 16:47:
0xe001ec2ef010 45 UTC+0000	UDPv6	:::0	*:*		3720	powershell.exe	2019-11-18 16:47:
0xe001ec485be0 45 UTC+0000	UDPv4	0.0.0.0:0	*:*		3720	powershell.exe	2019-11-18 16:47:
0xe001ec485be0 45 UTC+0000	UDPv6	:::0	*:*		3720	powershell.exe	2019-11-18 16:47:
0xe001ec7bd010 45 UTC+0000	UDPv4	0.0.0.0:0	*:*		3720	powershell.exe	2019-11-18 16:47:
0xe001ec932a80 14 UTC+0000	UDPv4	0.0.0.0:0	*:*		3004	powershell.exe	2019-11-18 16:55:
0xe001ece42360	UDPv4	0.0.0.0:0	*:*		3720	powershell.exe	2019-11-18 16:47:

Suspicious connections have originated from powershell.exe instances, all of which have connected to the same remote IP address on port 80.

Task 3. Hunt for process injection

When hunting for injection on a memory dump, Volatility's *malfind* plugin is the go to approach. Running the plugin returns that injection was detected in the following processes:

```
vol.py -f /root/Memory.vmem --profile=Win10x64 10586 malfind
```

```

root@attackdefense:~# vol.py -f /root/Memory.vmem --profile=Win10x64_10586 malfind
Volatility Foundation Volatility Framework 2.6.1
Process: MsMpEng.exe Pid: 2012 Address: 0x2a4614e0000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: PrivateMemory: 1, Protection: 6

0x2a4614e0000  20 00 00 00 e0 ff 0f 00 0c 00 00 00 01 00 07 00  .....
0x2a4614e0010  00 42 00 30 00 70 00 60 00 50 00 c0 00 d0 00 00  .B.0.p.`.P.....
0x2a4614e0020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0x2a4614e0030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....

0x614e0000 2000          AND [EAX], AL
0x614e0002 0000          ADD [EAX], AL
0x614e0004 e0ff          LOOPNZ 0x614e0005
0x614e0006 0f00c00      STR WORD [EAX+EAX]
0x614e000a 0000          ADD [EAX], AL
0x614e000c 0100          ADD [EAX], EAX
0x614e000e 07           POP ES
0x614e000f 0000          ADD [EAX], AL
0x614e0011 42           INC EDX
0x614e0012 0030          ADD [EAX], DH
0x614e0014 007000        ADD [EAX+0x0], DH
0x614e0017 60           PUSHA
0x614e0018 005000        ADD [EAX+0x0], DL
0x614e001b c000d0      ROL BYTE [EAX], 0xd0
0x614e001e 0000          ADD [EAX], AL
0x614e0020 0000          ADD [EAX], AL
0x614e0022 0000          ADD [EAX], AL
0x614e0024 0000          ADD [EAX], AL
0x614e0026 0000          ADD [EAX], AL
0x614e0028 0000          ADD [EAX], AL
0x614e002a 0000          ADD [EAX], AL
0x614e002c 0000          ADD [EAX], AL
0x614e002e 0000          ADD [EAX], AL
0x614e0030 0000          ADD [EAX], AL
0x614e0032 0000          ADD [EAX], AL
0x614e0034 0000          ADD [EAX], AL

```

- MsMpEng.exe (PID 2012) -- 2 injected memory regions
- Powershell.exe (PID 3720) -- 5 injected memory regions
- SearchUI.exe (PID 4784) -- 1 injected memory region
- Powershell.exe (PID 4740) -- 4 injected memory regions
- Powershell.exe (PID 3004) - 5 injected memory regions

It is normal for some processes to show signs of injection due to the way they operate, where in this case MsMpEng.exe and SearchUI.exe are false positives. PowerShell on the other hand should not have any signs of injection under normal operations. None of the injected memory regions begin with the "MZ" header though, which may indicate an injection of Windows executable file. Nonetheless, there may be multiple reasons for that, ranging from a tool that overwrites the injected memory regions as an anti-forensic technique after execution to prevent analysts from obtaining the injected code, to simply injection of raw shellcode instead of a whole executable.

Task 4. What can we conclude?

PowerShell processes have had suspicious internet connections and exhibit signs of injections. PowerShell's scripts are often still stored in the memory of a machine and using the "strings" utility, we can attempt to find certain patterns. For example, simply checking for "Invoke-" results in a wealth of information:

```
strings /root/Memory.vmem | grep "Invoke-"
```

```

root@attackdefense:~# strings /root/Memory.vmem | grep "Invoke-"
function Invoke-Mimikatz
    Function Invoke-CreateRemoteThread
        [IntPtr]$RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $LoadLibraryAAddr -ArgumentPtr $RImportDllPath
    Ptr -Win32Functions $Win32Functions
        $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs
        $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs -RemoteProcHandle $RemoteProcHandle
        $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
PowerShell:Invoke-E
    Function Invoke-CreateRemoteThread
        Invoke-Command -ScriptBlock $RemoteScriptBlock -ArgumentList @($PEBytes64, $PEBytes32, "Void", 0, "", $ExeArgs)
        Invoke-Command -ScriptBlock $RemoteScriptBlock -ArgumentList @($PEBytes64, $PEBytes32, "Void", 0, "", $ExeArgs) -ComputerName $ComputerName
    }Invoke-Mimikatz -DumpCreds;
function Invoke-Mimikatz
    Function Invoke-MemoryFreeLibrary
        $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
    Function Invoke-MemoryLoadLibrary
    Function Invoke-MemoryLoadLibrary
        $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
        [IntPtr]$RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $LoadLibraryAAddr -ArgumentPtr $RImportDllPath
    Ptr -Win32Functions $Win32Functions
        $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs
        $PELoadedInfo = Invoke-MemoryLoadLibrary -PEBytes $PEBytes -ExeArgs $ExeArgs -RemoteProcHandle $RemoteProcHandle
        $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $VoidFuncAddr -Win32Functions $Win32Functions
        Invoke-MemoryFreeLibrary
    Function Invoke-MemoryLoadLibrary
        "n[*] Running Invoke-AllChecks"
        $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $VoidFuncAddr -Win32Functions $Win32Functions
        Invoke-MemoryFreeLibrary -PEHandle $PEHandle
        Invoke-Command -ScriptBlock $RemoteScriptBlock -ArgumentList @($PEBytes64, $PEBytes32, "Void", 0, "", $ExeArgs)
        Invoke-Command -ScriptBlock $RemoteScriptBlock -ArgumentList @($PEBytes64, $PEBytes32, "Void", 0, "", $ExeArgs) -ComputerName $ComputerName
    }Invoke-Mimikatz -DumpCreds;
G:Invoke-Y
    'Invoke-Pester',
    'Invoke-Mock',
        $RThreadHandle = Invoke-CreateRemoteThread -ProcessHandle $RemoteProcHandle -StartAddress $RSCAddr -Win32Functions $Win32Functions
    Function Invoke-MemoryFreeLibrary

```

From the very first lines, we can conclude that a compromise has occurred due to the presence of the string "Invoke-Mimikatz".

